

Implementação

1. Criar a Interface Base

Define o contrato para todas as especificações.

```
public interface ISpecification<T>
{
    bool IsSatisfiedBy(T entity);
    ISpecification<T> And(ISpecification<T> other);
    ISpecification<T> Or(ISpecification<T> other);
    ISpecification<T> Not();
}
```

2. Implementar a Classe Base

Fornece a implementação básica para combinar especificações.

```
public abstract class Specification<T> : ISpecification<T>
{
    public abstract bool IsSatisfiedBy(T entity);

    public ISpecification<T> And(ISpecification<T> other)
    {
        return new AndSpecification<T>(this, other);
    }

    public ISpecification<T> Or(ISpecification<T> other)
    {
        return new OrSpecification<T>(this, other);
    }

    public ISpecification<T> Not()
    {
        return new NotSpecification<T>(this);
    }
}
```

3. Criar Especificações Compostas

Combina especificações usando lógica booleana.

```
public class AndSpecification<T> : Specification<T>
{
    private readonly ISpecification<T> _left;
    private readonly ISpecification<T> _right;

    public AndSpecification(ISpecification<T> left, ISpecification<T>
right)
    {
        _left = left;
        _right = right;
    }

    public override bool IsSatisfiedBy(T entity)
    {
        return _left.IsSatisfiedBy(entity) &&
_right.IsSatisfiedBy(entity);
    }
}

public class OrSpecification<T> : Specification<T>
{
    private readonly ISpecification<T> _left;
    private readonly ISpecification<T> _right;

    public OrSpecification(ISpecification<T> left, ISpecification<T>
right)
    {
        _left = left;
        _right = right;
    }

    public override bool IsSatisfiedBy(T entity)
    {
        return _left.IsSatisfiedBy(entity) ||
_right.IsSatisfiedBy(entity);
    }
}

public class NotSpecification<T> : Specification<T>
{
    private readonly ISpecification<T> _specification;

    public NotSpecification(ISpecification<T> specification)
    {
        _specification = specification;
    }
}
```

```

    }

    public override bool IsSatisfiedBy(T entity)
    {
        return !_specification.IsSatisfiedBy(entity);
    }
}

```

4. Criar Especificações Concretas

Encapsula regras específicas de validação.

```

public class PriceSpecification : Specification<Product>
{
    private readonly decimal _minPrice;

    public PriceSpecification(decimal minPrice)
    {
        _minPrice = minPrice;
    }

    public override bool IsSatisfiedBy(Product product)
    {
        return product.Price >= _minPrice;
    }
}

public class CategorySpecification : Specification<Product>
{
    private readonly string _category;

    public CategorySpecification(string category)
    {
        _category = category;
    }

    public override bool IsSatisfiedBy(Product product)
    {
        return product.Category == _category;
    }
}

```

5. Criar a Entidade

Define o objeto a ser filtrado ou validado.

```
public class PriceSpecification : Specification<Product>
{
    private readonly decimal _minPrice;

    public PriceSpecification(decimal minPrice)
    {
        _minPrice = minPrice;
    }

    public override bool IsSatisfiedBy(Product product)
    {
        return product.Price >= _minPrice;
    }
}

public class CategorySpecification : Specification<Product>
{
    private readonly string _category;

    public CategorySpecification(string category)
    {
        _category = category;
    }

    public override bool IsSatisfiedBy(Product product)
    {
        return product.Category == _category;
    }
}

public class Product
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string Category { get; set; }
}
```

6. Usar as Especificações

```
class Program
{
    static void Main(string[] args)
    {
        // Criar lista de produtos
        var products = new List<Product>
        {
            new Product { Name = "Laptop", Price = 1500, Category =
"Electronics" },
            new Product { Name = "Mouse", Price = 50, Category =
"Electronics" },
            new Product { Name = "Shampoo", Price = 10, Category =
"Beauty" },
            new Product { Name = "Keyboard", Price = 100, Category =
"Electronics" }
        };

        // Criar especificações
        var priceSpec = new PriceSpecification(100);
        var categorySpec = new CategorySpecification("Electronics");

        // Combinar especificações
        var spec = priceSpec.And(categorySpec);

        // Filtrar produtos
        var filteredProducts = products.Where(p =>
spec.IsSatisfiedBy(p)).ToList();

        // Exibir produtos filtrados
        foreach (var product in filteredProducts)
        {
            Console.WriteLine($"{product.Name} - {product.Price:C} -
{product.Category}");
        }
    }
}
```

Saída Esperada

```
Laptop - $1,500.00 - Electronics  
Keyboard - $100.00 - Electronics
```

Vantagens do Specification Pattern

1. **Reuso:** Especificações podem ser reutilizadas em diferentes partes do sistema.
2. **Modularidade:** Regras de negócios são encapsuladas, tornando o código mais limpo.
3. **Composição:** Especificações podem ser combinadas facilmente (And, Or, Not).

Quando evitar?

- Se as regras de validação são simples e não precisam de reutilização.
- Se o número de especificações for muito pequeno, tornando o padrão desnecessário.

Resumo

O **Specification Pattern** é ideal para encapsular regras de negócios ou validações complexas de maneira reutilizável e modular. Ele é altamente flexível e se destaca em sistemas que precisam de critérios dinâmicos ou frequentemente alterados. □